

Reglas Firestore Finales Recomendadas

Entrega Final · Dispositivos Móviles · 2026

09-REGLAS-FIRESTORE-FINALES

AUTORES

Andrés Botero

Tecnología en Desarrollo de Software

Juan Camilo Triana

Tecnología en Desarrollo de Software

PROYECTO

OutfitCatalog · ATELIER Fashion Catalog

ASIGNATURA

Dispositivos Móviles

FECHA

Junio 2026

VERSIÓN

1.0.0

Contenido

Reglas Firestore Finales Recomendadas

Nota sobre usuario administrador

Estas reglas parten de las reglas actuales que compartiste y agregan la nueva coleccion analyticsEvents para KPIs. Tambien mantienen purchaseRequests, users, garments y looks.

```
JS
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    function signedIn() {
      return request.auth != null;
    }

    function isOwner(userId) {
      return signedIn() && request.auth.uid == userId;
    }

    function hasProfile() {
      return signedIn() &&
        exists(/databases/{database}/documents/users/{request.auth.uid});
    }

    function myRole() {
      return hasProfile()
        ? get(/databases/{database}/documents/users/{request.auth.uid}).data.role
        : "user";
    }

    function isAdmin() {
      return myRole() == "admin";
    }

    function isVendorOrAdmin() {
      return myRole() == "vendor" || myRole() == "admin";
    }

    function validPurchaseRequest(data) {
      return data.buyerId is string
        && data.buyerName is string
        && data.buyerEmail is string
        && data.vendorId is string
        && data.vendorName is string
        && data.source in ["garment", "look"]
        && data.sourceId is string
        && data.sourceName is string
        && data.items is list
        && data.total is number
        && data.status in ["pending", "contacted", "reserved", "sold", "cancelled"]
        && data.message is string
        && data.createdAt is string
        && data.updatedAt is string;
    }

    function validAnalyticsEvent(data) {
      return data.name is string
        && data.createdAt is string
        && data.sessionId is string
        && data.platform is string
        && data.params is map;
    }

    match /users/{userId} {
      allow create: if isOwner(userId)
        && request.resource.data.email is string
        && request.resource.data.name is string
        && request.resource.data.role in ["user", "vendor"];
```

```

allow read: if isOwner(userId) || isAdmin();
allow update: if (isOwner(userId) && request.resource.data.userId == request.resource.data.userId) || isAdmin();

allow delete: if isAdmin();
}

match /garments/{garmentId} {
  allow read: if true;
  allow create, update, delete: if isVendorOrAdmin();
}

match /looks/{lookId} {
  allow read: if isOwner(resource.data.userId) || isAdmin();
  allow create: if isOwner(request.resource.data.userId);
  allow update, delete: if isOwner(resource.data.userId) || isAdmin();
}

match /purchaseRequests/{requestId} {
  allow create: if signedIn()
    && request.auth.uid == request.resource.data.buyerId
    && validPurchaseRequest(request.resource.data);

  allow read: if signedIn() && (
    request.auth.uid == resource.data.buyerId ||
    request.auth.uid == resource.data.vendorId ||
    isAdmin()
  );

  allow update: if signedIn()
    && (request.auth.uid == resource.data.vendorId || isAdmin())
    && request.resource.data.diff(resource.data).affectedKeys().hasOnly(["status",
"updatedAt"])
    && request.resource.data.status in ["pending", "contacted", "reserved", "sold",
"cancelled"];

  allow delete: if signedIn() && (
    request.auth.uid == resource.data.buyerId ||
    request.auth.uid == resource.data.vendorId ||
    isAdmin()
  );
}

match /analyticsEvents/{eventId} {
  allow create: if signedIn()
    && validAnalyticsEvent(request.resource.data)
    && (
      !request.resource.data.keys().hasAny(["userId"])
      || request.resource.data.userId == request.auth.uid
    );

  allow read: if isAdmin();
  allow update, delete: if false;
}
}
}

```

Nota sobre usuario administrador

Las reglas mantienen bloqueada la creacion de administradores desde la app (users solo permite user y vendor). Esto es correcto por seguridad. El usuario admin debe crearse mediante cuenta de servicio usando:

BASH

```
npm run seed:admin -- --service-account "C:\ruta\service-account.json"
```

Por defecto crea o actualiza:

TEXT

Email: admin@outfit.test
Password: Admin123!
Rol: admin
Telefono: 3104221496

Para pruebas actuales de contacto por WhatsApp, los vendedores generados por seed usan:

TEXT

Telefono vendedores: +573104221496